

AU/NZ TRACTOR COMPARISON TOOL

GitHub Pages Migration and Maintainable Data Architecture



	Document purpose A single source of truth for the project team and for AI coding assistants. This document defines what is being built, why it is being built, how the data and application will work, and the rules that future changes must follow.
Project owner	Archie Ventrella
Audience	Project contributors, reviewers, product specialists and AI coding assistants
Delivery model	Static web application hosted through GitHub Pages
Primary data source	data-source/machines.csv
Document version	1.0 10 September 2026

John Deere green #367C2B • Yellow #FFD700

1. Executive summary

The existing AU/NZ Tractor Comparison Tool is a browser-based comparison application containing a large set of tractor specification records. The current implementation is functional, but machine data is embedded in a self-contained HTML file. This makes frequent corrections, new model-year releases and new manufacturer additions harder to manage than they need to be.

The new version will separate the application from its data. The front end will be hosted on GitHub Pages. A single UTF-8 CSV file will become the human-maintained master data source. An automated GitHub Actions workflow will validate the CSV, transform it into optimised JSON files, and publish the updated site. Contributors will update rows in the CSV rather than editing application code or generated JSON.

Target outcome

A maintainer can add a model, add a model year, correct a specification, introduce a manufacturer or change publication status by editing one row in machines.csv. After review and merge, the automation rebuilds the data and republishes the live site without a manual HTML rebuild.

1.1 The project in one sentence

Build a maintainable, data-driven tractor comparison website where the CSV is the single source of truth and GitHub automatically handles validation, JSON creation and deployment.

1.2 Key decisions already made

- GitHub Pages will host the static website.
- The site will use an unlisted URL distributed through the intended internal channel. This is link-controlled distribution, not authenticated access.
- The application will contain no confidential customer data, pricing, margins, credentials or confidential internal benchmarking.
- One master CSV file will be the only machine-data file contributors edit.
- Generated JSON files will be build outputs and must not be manually maintained.
- Every market and model year will be represented by a separate machine record.
- Existing horsepower-band relationship logic will remain, with the option to add curated relationships later.
- Machine images will initially be stored in the GitHub repository using predictable filenames.

2. Purpose, goals and boundaries

2.1 Business purpose

The tool brings published machine specifications into one consistent interface so John Deere employees and approved dealer personnel can identify broadly comparable tractors, filter machines, review specification differences and export information for internal preparation and product familiarisation.

2.2 Project goals

Goal	Definition of success
Easy maintenance	A contributor updates machine content by editing CSV rows without changing JavaScript.
Frequent releases	Small corrections and larger model-year updates follow the same controlled workflow.
Scalable catalogue	New models, years, series and makes appear from data rather than hard-coded lists.

Goal	Definition of success
Reliable publishing	Invalid data blocks deployment and valid changes publish automatically after merge.
Traceability	Git history shows who changed the data, what changed and why.
Consistent user experience	Filters, relationships, comparisons and exports continue to work across the expanded data set.
AI maintainability	An AI coding assistant can understand the system from this document, the repository README and machine-readable validation rules.

2.3 In scope for the first GitHub version

- Static GitHub Pages front end
- CSV-to-JSON build process
- Automated data validation
- Dynamic manufacturer, model-year and filter generation
- Current comparison and horsepower relationship functions
- Machine images and placeholders
- Release/build information
- Unlisted URL distribution
- GitHub pull request review and automated deployment

2.4 Out of scope for the first version

- A live relational database
- Browser-based editing of machine records
- Public user registration
- Dealer single sign-on
- Fine-grained application roles
- Pricing or commercial comparisons
- Customer data
- Confidential internal benchmarking
- AI-generated selling recommendations
- Automated extraction of specifications from manufacturer websites

Important boundary

An unlisted GitHub Pages URL is not an authentication system. Anyone who obtains the link may be able to open it. This model is suitable only while the published content is approved for link-controlled access and contains no sensitive information.

3. Solution architecture

3.1 End-to-end flow

```

Contributor edits data-source/machines.csv
    |
    v
Pull request or reviewed change
    |
    v
GitHub Actions validation
    |
    +-- validation fails --> deployment stops with row-level errors
    |
    v
CSV transformation and JSON generation
    |
    v
Static-site build and GitHub Pages deployment
    |
    v
Users load the refreshed application from the unlisted page URL
  
```

3.2 Proposed repository structure

```

tractor-comparison-tool/
├── README.md
├── index.html
├── data-source/
│   └── machines.csv
├── src/
│   ├── css/
│   │   └── styles.css
│   └── js/
│       ├── app.js
│       ├── data-loader.js
│       ├── filters.js
│       ├── relationships.js
│       ├── comparison.js
│       └── exports.js
├── assets/
│   └── images/machines/<manufacturer>/
├── scripts/
│   ├── validate-data.js
│   └── build-data.js
├── dist/                                     # generated; not manually edited
│   ├── index.html
│   ├── data/
│   │   ├── machines.json
│   │   ├── manufacturers.json
│   │   ├── filter-options.json
│   │   ├── model-years.json
│   │   └── build-info.json
│   └── assets/
├── .github/workflows/
│   └── build-and-deploy.yml
  
```

3.3 Component responsibilities

Component	Responsibility	Must not do
<code>machines.csv</code>	Human-maintained source of machine records and control fields.	Contain formulas, merged cells or hidden application logic.

Component	Responsibility	Must not do
validate-data.js	Check schema, IDs, required values, controlled values, references and data quality.	Silently repair serious errors.
build-data.js	Clean values, convert types, split tags, derive indexes and write JSON.	Modify the source CSV.
Generated JSON	Provide fast, structured runtime data for the site.	Be edited by contributors.
Front-end JavaScript	Load JSON, build filters, calculate relationships, compare and export.	Contain hard-coded manufacturers or machine records.
GitHub Actions	Run checks, create deployment artifacts and publish Pages.	Deploy if mandatory validation fails.
GitHub Pages	Serve the static site over HTTPS.	Be treated as a secure database or identity provider.

4. Master CSV data design

The initial CSV was transformed from PublicSpecComparisonDataMaster_updated.xlsx. The prepared source contains 316 valid machine rows and 43 columns, including ten new control and maintenance columns. The CSV should be saved as UTF-8 with a header row and one machine record per row.

4.1 Record grain

One row means one machine specification record

The unique grain is manufacturer + model/configuration + model year + market. A new model year is a new row. Do not overwrite an older valid record simply because a newer model year appears.

4.2 Control and maintenance columns

Column	Purpose	Rule
machine_id	Permanent technical identifier used by the site and relationships.	Must be unique and stable after publication.
published	Controls whether the record is included in generated public data.	TRUE or FALSE only.
lifecycle_status	Describes the record lifecycle.	Blank initially; later controlled values such as Upcoming, Current, Discontinued, Archived.
market	Identifies specification market.	AU, NZ, AU/NZ, UK or US unless schema is deliberately expanded.
image_filename	Relative machine image filename.	Blank allows a placeholder.
image_alt_text	Accessible description of the machine image.	Required when image_filename is populated.
source_url	Plain source URL for specification traceability.	Recommended for published records.
source_title	Human-readable source title.	Recommended for published records.
last_reviewed_date	Date the row was last reviewed.	ISO format YYYY-MM-DD when populated.
notes	Maintenance context, assumptions or caveats.	Not for credentials or sensitive data.

4.3 Identity rules

machine_id pattern:
 <manufacturer-and-model-slug>-<model-year>-<market>

Examples:
 john-deere-8r-410-2025-au
 massey-ferguson-5s-115-2025-au
 case-ih-optum-390-2025-uk

- Use lowercase ASCII letters, numbers and hyphens only.
- Do not include redundant brand tokens. Example: use massey-ferguson-5s-115, not massey-ferguson-mf-5s-115.
- Do not recycle an ID for a different machine.

- Do not change an ID merely to improve wording after the record is referenced. If an ID must change, update all references in the same change.
- The visible machine name can retain trademark symbols and punctuation even though the ID cannot.

4.4 Missing-value rules

Contributors should leave unknown values blank. The build process will interpret legacy values such as NA, N/A, No Data and ~ as missing and output JSON null. The user interface will present one consistent missing-value label.

4.5 Numeric versus descriptive fields

Type	Examples	Build treatment
Clean numeric	model_year, rated_hp, max_hp, max_hp_with_ipm, max_torque_nm	Convert to JSON numbers when valid.
Tag list	transmission_filter_tags, rear_pto_filter_tags	Split on the pipe character into arrays.
Descriptive specification	fuel_capacity_l, rear_hitch_capacity_kg, unladen_weight_kg when options are included	Preserve source text if the value contains options, qualifiers or multiple configurations.
Controlled value	published, market, lifecycle_status, transmission_class	Validate against allowed values.
Free maintenance text	notes, source_title, image_alt_text	Trim whitespace and preserve wording.

4.6 Current schema fields

Group	Columns
Identity and control	machine_id, published, lifecycle_status, market
Presentation and sources	image_filename, image_alt_text, source_url, source_title, last_reviewed_date, notes
Machine identity	machine, manufacturer, country_of_manufacture, model_year
Power	rated_hp, max_hp, max_hp_with_ipm, max_torque_nm
Transmission and filters	transmission, transmission_filter_tags, transmission_class
Speed and dimensions	top_speed_filter, cylinders_filter, top_speed_kmh, wheelbase_mm
Weight	unladen_weight_kg, max_permissible_weight_40_kmh_kg, max_permissible_weight_50_kmh_kg, max_permissible_weight_60_kmh_kg
Hydraulics and hitch	rated_hydraulic_flow_lpm, max_hydraulic_flow_lpm, max_scvs, rear_hitch_capacity_kg, front_hitch_capacity_kg
Engine and capacities	engine_capacity_l, number_of_cylinders, engine_manufacturer, cab_suspension, fuel_capacity_l, engine_stage_tier, adblue_tank_l
PTO	rear_pto_option, rear_pto_filter_tags

5. Automated validation and transformation

5.1 Validation philosophy

The automation should be strict about records that could break the application or create ambiguous identity. It should be informative rather than obstructive for optional specifications. Errors stop deployment. Warnings are visible in the workflow summary but do not necessarily stop publication.

5.2 Errors that stop deployment

- Missing machine_id, machine, manufacturer, model_year or market
- Duplicate machine_id
- Duplicate manufacturer + machine + model_year + market combination
- Invalid published value
- Invalid model year
- Published record with no usable max_hp when horsepower relationships require it
- Malformed pipe-separated tags
- Unexpected or missing CSV headers
- A machine ID containing spaces, uppercase characters or unsupported symbols
- Image filename present but image_alt_text missing
- Generated JSON that is invalid or empty

5.3 Warnings that should be reported

- No source URL or source title
- No image
- No torque figure
- No lifecycle status
- A specification value outside a configurable expected range
- A UK or US specification included in an AU-focused data set
- Legacy missing-value text detected and normalised
- A clean numeric value rounded to remove spreadsheet floating-point noise

5.4 Transformation rules

Input condition	Generated result
Blank, NA, N/A, No Data or ~	null
429.000000000000011	429
4.4000000000000004	4.4
Semi-powershift CVT / IVT / EVT	["Semi-powershift", "CVT / IVT / EVT"]
540E 540 1000E 1000	["540E", "540", "1000E", "1000"]
John Deere with trailing space	John Deere
Published = FALSE	Record excluded from the public machines.json but available to validation reports
New manufacturer value	Manufacturer automatically included in generated manufacturer and filter indexes

5.5 Required generated files

File	Contents
machines.json	Published, cleaned machine objects used by the application.
manufacturers.json	Unique manufacturer list, counts or optional display metadata derived from published records.
filter-options.json	Available transmission classes, transmission tags, PTO tags, speed values, cylinder values, markets and statuses.
model-years.json	Unique published model years, sorted newest to oldest.
build-info.json	Build timestamp, data version, published machine count and commit reference.
validation-report.json	Machine-readable errors and warnings retained as a workflow artifact, not necessarily published to users.

6. Application behaviour

6.1 Data loading

- The application loads generated JSON using relative paths so it works on the GitHub Pages project URL.
- The application displays a clear error state if data cannot be loaded.
- The application must not contain embedded fallback machine records that can become stale.
- The application should include a cache-busting build version or hashed asset names so users receive updated data after deployment.

6.2 Filters

Manufacturer, model year, transmission, speed, cylinder and PTO filter options must be derived from generated data. Adding a manufacturer or year to the CSV should make it available without a code change.

6.3 Relationship search

The initial relationship method remains Max HP based. The selected machine is pinned and other published records are matched within the user-selected percentage band. Results are sorted by absolute horsepower difference unless the user applies another sort.

```
lower_bound = selected.max_hp × (1 - percentage / 100)
upper_bound = selected.max_hp × (1 + percentage / 100)
match when candidate.max_hp is between lower_bound and upper_bound
```

6.4 One-to-one comparison

- Allow one selected machine and one comparison machine.
- Show specification labels consistently, independent of CSV header wording.
- Calculate numeric deltas only when both values are suitable clean numbers.
- Preserve raw descriptive text where options or qualifiers make a numeric delta misleading.
- Provide copy, print or CSV export functions using the current visible comparison.

6.5 Images

```
assets/images/machines/<manufacturer-slug>/<machine_id>.webp
```

Example:
assets/images/machines/john-deere/john-deere-8r-410-2025-au.webp

- Use approved images only.
- Prefer compressed WebP files for normal catalogue images.
- Use image_alt_text from the CSV.
- Display a neutral manufacturer/model placeholder when no approved image is available.
- Do not make image availability a requirement for publishing an otherwise valid machine record.

6.6 Release information

The interface should show a compact data release indicator using build-info.json, including the last build date and a version or commit reference. This helps users understand whether the catalogue has been refreshed without exposing technical details unnecessarily.

7. Data maintenance standard operating procedure

7.1 Add a new machine

1. Create a new branch from main.
2. Open data-source/machines.csv using a CSV-safe editor or Excel.
3. Add one complete row.
4. Create a unique machine_id using the documented pattern.
5. Set published to FALSE until the record has been reviewed.
6. Populate the machine, manufacturer, year, market, Max HP and available specifications.
7. Add source and review information where available.
8. Add an approved image using the machine_id filename, or leave the image fields blank.
9. Save as UTF-8 CSV without formulas or extra worksheets.
10. Commit the change and open a pull request.
11. Resolve automated errors and review warnings.
12. After review, set published to TRUE and merge.

7.2 Add a new model year

13. Copy the previous applicable model-year row.
14. Assign a new machine_id with the new year.
15. Update model_year.
16. Update only specifications supported by the new source material.
17. Do not automatically carry forward unverified values. If an unchanged field is not confirmed, retain it only when the maintenance policy explicitly permits carry-forward and note the basis.
18. Keep the earlier model-year row. Change its lifecycle status later if appropriate.
19. Update source, review and image fields.
20. Follow the normal pull request and publishing process.

7.3 Add a new manufacturer

21. Add machine rows using one consistent manufacturer spelling.
22. Use a stable manufacturer slug in IDs and image folders.
23. Add the manufacturer image folder if images will be used.
24. Do not add the manufacturer to JavaScript filter lists. The build derives it automatically.
25. Review sorting, filter display and placeholder behaviour in the deployment preview.

7.4 Correct an existing specification

26. Locate the row by machine_id rather than only by visible name.
27. Change only the affected fields.
28. Update source information and last_reviewed_date.
29. Explain non-obvious changes in notes or the pull request description.
30. Do not change machine_id unless the identity itself is wrong and all references can be updated together.

7.5 Remove a machine from normal use

Do not delete a valid historical record by default. Set published to FALSE to remove it from the live data, or use a future lifecycle status such as Discontinued or Archived while retaining it for relevant comparisons. Permanent deletion should be reserved for duplicate or invalid records and explained in the change history.

8. GitHub workflow, deployment and security

8.1 Branch model

main	protected publishing branch
feature/add-2027-8r	model-year update
fix/case-ih-optum-weight	specification correction
feature/add-brand	new manufacturer update

8.2 Pull request expectations

- Describe the affected machines and reason for the change.
- Include the source used for the update when practical.
- Do not bundle unrelated machine changes with application-code changes unless required.
- Review the automated validation summary.
- Use at least one human review for broad data releases or structural changes.
- Merge only after mandatory checks pass.

8.3 Deployment pipeline

Stage	Action	Failure behaviour
Checkout	Retrieve repository content for the approved commit.	Stop if repository content cannot be retrieved.
Install	Install locked build dependencies.	Stop on dependency error.
Validate	Check CSV schema and data rules.	Stop on errors; publish report.
Build data	Generate JSON and build metadata.	Stop if output is invalid.
Build site	Copy HTML, CSS, JS, images and generated data to dist.	Stop if expected files are missing.
Smoke test	Confirm the built page and generated data load successfully.	Stop on failed checks.
Deploy	Upload the Pages artifact from the approved main commit.	Leave previous successful deployment intact when possible.

8.4 Security position

Approved access model for this phase

The site is unlisted and the URL is distributed through the intended internal channel. The URL may be forwarded. The application must therefore be safe if viewed by anyone who obtains the link. Client-side access codes, referrer checks and JavaScript passwords must not be presented as secure authentication.

- Enforce HTTPS through GitHub Pages.
- Keep the source repository under the approved Deere GitHub organisation and permissions model.
- Do not store secrets in HTML, JavaScript, CSV, JSON, repository files or Git history.
- Do not store confidential data in the application.
- Do not connect the browser directly to a database.
- Do not include write-capable tokens in the front end.
- Use dependency pinning and automated dependency/security scanning where available.

- If stronger access control becomes necessary, reassess the hosting design rather than adding cosmetic client-side gates.

9. Testing and acceptance criteria

9.1 Data tests

Test	Expected result
Add a valid new row	Validation passes and the machine appears in generated JSON after publication.
Add a new manufacturer	Manufacturer filter updates without code changes.
Add a new model year	Year filter updates and earlier year remains available.
Duplicate an ID	Deployment is blocked with a row-level error.
Enter an invalid model year	Deployment is blocked.
Use pipe-separated tags	Generated JSON contains arrays with trimmed tags.
Use NA or No Data	Generated value becomes null.
Enter 429.00000000000011	Generated clean numeric value becomes 429.
Set published to FALSE	Record is excluded from public machine output.
Provide an image without alt text	Validation reports an error or configured blocking issue.

9.2 Functional tests

- Site loads successfully from the project Pages path.
- Selected machine remains pinned in relationship results.
- Percentage relationship search returns only eligible published records.
- Manufacturer, year, transmission, speed, cylinders and PTO filters combine correctly.
- One-to-one comparison displays values and safe numeric deltas.
- Copy, print and export functions reflect the current selection.
- Missing values display consistently.
- Images and placeholders render correctly.
- Build information reflects the deployed data.
- The site remains usable on common desktop screen sizes and is accessible by keyboard.

9.3 First release acceptance criteria

Release is acceptable when

All 316 migrated records can be transformed without duplicate IDs; the existing comparison behaviour works against generated JSON; adding a test model and test manufacturer requires only CSV changes; invalid source data blocks deployment; and the live Pages site updates from an approved main-branch merge.

10. Implementation plan

Phase	Deliverables	Exit condition
1. Repository foundation	Repository structure, README, machines.csv, branch protection and Pages configuration.	Source CSV is committed and ownership is clear.
2. Data pipeline	Schema definition, validation script, build script and generated JSON contracts.	All migrated records build successfully and deliberate errors fail clearly.
3. Front-end migration	Existing interface split into HTML, CSS and modules; embedded data removed; JSON loader added.	Current comparison functions work with generated data.
4. Automation	GitHub Actions workflow, build artifact, smoke tests and Pages deployment.	Approved main changes publish automatically.
5. Visual enhancement	Machine image support, placeholders, release indicator and responsive refinements.	Visual experience is consistent across representative machines.
6. Handover	Contributor guide, AI context, maintenance examples and release checklist.	Another contributor can add and publish a model without code changes.

10.1 Recommended first coding milestone

The first milestone should not rebuild the complete interface. It should prove the data contract and automation with a small local page that loads the generated machines.json. Once the CSV schema and build outputs are stable, migrate the existing comparison interface onto them. This avoids writing the interface twice if the schema changes.

10.2 Immediate implementation sequence

31. Create the GitHub repository and commit this documentation and machines.csv.
32. Create schema/configuration that defines every column, type, required status and controlled values.
33. Build validate-data.js.
34. Build build-data.js and generate the five required JSON files.
35. Add automated tests for representative records and known edge cases.
36. Create the GitHub Actions workflow and verify that bad data blocks deployment.
37. Refactor the existing HTML into separated source files.
38. Replace embedded machine data with the generated JSON loader.
39. Run functional regression tests against the current tool.
40. Enable GitHub Pages deployment from the workflow.

11. Instructions for AI coding assistants

Use this section as the AI operating brief

An AI assistant should read this document, README.md, the CSV header and validation configuration before changing code. If a requested change conflicts with these rules, the assistant should identify the conflict rather than silently redesigning the architecture.

11.1 Non-negotiable rules

- Treat data-source/machines.csv as the single source of truth.
- Never manually edit generated JSON as the solution to a data problem.
- Never hard-code manufacturer, model-year or filter-value lists in the front end.
- Never expose secrets or credentials in the static application.
- Never infer missing tractor specifications. Preserve blanks as unknown.
- Never overwrite a previous valid model-year row when adding a new year.
- Never alter a published machine_id without checking and updating every reference.
- Preserve source wording for complex specification fields unless a documented normalisation rule applies.
- Use ASCII punctuation in code and configuration where practical, while preserving trademark and model characters in display data.
- Make changes small, reviewable and testable.

11.2 Preferred AI task format

```
Task objective:
Affected files:
Data-contract impact:
Implementation steps:
Validation/tests added or updated:
Backward-compatibility considerations:
Security considerations:
Result and any unresolved limitations:
```

11.3 AI completion checklist

- Did the change preserve CSV as the source of truth?
- Did the change avoid hard-coded catalogue values?
- Did the change add or update tests?
- Did the change preserve existing filters and comparison logic?
- Did the change avoid inventing specifications or lifecycle status?
- Did the change keep secrets out of the browser and repository?
- Did the change work when hosted below a GitHub Pages project path?
- Did the change update documentation if the schema or workflow changed?

12. Ownership, governance and future options

12.1 Suggested responsibilities

Responsibility	Suggested owner type
Product scope and approved audience	Business owner or product sponsor
Machine specification accuracy	Relevant product specialist or nominated reviewer
CSV schema and validation rules	Technical maintainer
Application code and deployment workflow	Technical maintainer or development team
Images and usage approval	Marketing or product image owner
Release approval	Project owner with required reviewers
Security model review	Appropriate Deere security or platform owner if scope changes

12.2 Change categories

Change type	Typical review
Single data correction	Data review and automated checks
New machine or model year	Data review, source review and automated checks
New manufacturer	Data review plus user-interface preview
CSV schema change	Technical review, migration plan and documentation update
Comparison logic change	Technical and product-behaviour review
Audience or security change	Architecture and security review before implementation

12.3 Future enhancements

- Curated competitor relationships in a separate CSV
- A browser-based admin interface backed by an approved API and database
- Dealer identity integration
- Structured source catalogue and per-field provenance
- Bulk import templates by manufacturer
- Automated preview deployments for pull requests
- Visual change reports showing added, changed and removed machines
- Lifecycle dashboards for records needing review
- Automated image-size and image-rights checks
- Expansion to additional machine categories using the same build framework

Future architecture trigger

Move beyond a pure GitHub Pages solution when the tool needs secure user authentication, browser-based writes, private data, per-user permissions, or server-side business logic. Those needs require an approved backend rather than additional client-side JavaScript.

13. Glossary

Term	Meaning
Source of truth	The authoritative editable data source. For this project, machines.csv.
Generated artifact	A file created by automation and not manually maintained, such as machines.json.
Schema	The defined columns, data types, required values and validation rules.
Machine ID	Permanent identifier for a machine record.
Record grain	What one row represents: one model/configuration, model year and market.
GitHub Actions	Repository automation used for validation, building and deployment.
GitHub Pages	Static website hosting for the built application.
Pull request	A proposed repository change reviewed before merging.
Published record	A CSV row with published = TRUE that is included in the live site data.
Unlisted URL	A link not broadly advertised. It is not the same as authenticated access.
Regression test	A check that confirms existing behaviour still works after a change.

14. Initial project checklist

- ☐ Create approved GitHub repository
- ☐ Add this project documentation
- ☐ Add data-source/machines.csv
- ☐ Add README.md with quick-start instructions
- ☐ Define schema and controlled values
- ☐ Create validation script
- ☐ Create CSV-to-JSON build script
- ☐ Create automated tests
- ☐ Generate initial JSON successfully
- ☐ Refactor existing HTML into project files
- ☐ Connect interface to generated JSON
- ☐ Add image structure and placeholder
- ☐ Create GitHub Actions workflow
- ☐ Configure GitHub Pages
- ☐ Test unlisted-link distribution
- ☐ Complete regression testing
- ☐ Publish first controlled release
- ☐ Document the first real model-year update

Appendix A. Example machine row and JSON

A.1 CSV example

```
machine_id,published,lifecycle_status,market,machine,manufacturer,model_year,rated_hp,max_hp,transmission_filter_tags,rear_pto_filter_tags
massey-ferguson-5s-115-2025-au,TRUE,,AU,Massey Ferguson 5S.115,Massey Ferguson,2025,115,115,Semi-powershift,540E|540|1000
```

A.2 Generated JSON example

```
{
  "id": "massey-ferguson-5s-115-2025-au",
  "published": true,
  "lifecycleStatus": null,
  "market": "AU",
  "machine": "Massey Ferguson 5S.115",
  "manufacturer": "Massey Ferguson",
  "modelYear": 2025,
  "ratedHp": 115,
  "maxHp": 115,
  "transmissionFilterTags": ["Semi-powershift"],
  "rearPtoFilterTags": ["540E", "540", "1000"]
}
```

Appendix B. Suggested pull request template

```
## Change summary
Describe the models, model years or application functions changed.

## Reason
Explain why the change is required.

## Source and review
List the relevant public source and reviewer where applicable.

## Data changes
- Added:
- Updated:
- Unpublished or removed:

## Checks
- [ ] CSV validation passes
- [ ] Generated JSON reviewed
- [ ] Filters and comparison tested
- [ ] Images and alt text reviewed
- [ ] No sensitive data or secrets added
- [ ] Documentation updated if architecture or schema changed
```

Appendix C. Documentation sources and status

This project document consolidates the agreed design from the working discussion and the current source artefacts: PublicSpecComparisonDataMaster_updated.xlsx, the prepared machines.csv, the current self-contained comparison tool, and the draft project statement of work. GitHub Pages and GitHub Actions implementation details should be kept aligned with the current GitHub documentation when the repository workflow is created.

Living document

Update this document whenever the CSV schema, generated JSON contract, security model, deployment workflow or ownership model changes. Minor machine-data updates do not require a document revision unless they introduce a new rule.